COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	21/10/2026	**DATE SUBMITTED:**	21/10/2026
NAME:	Seth Carlo T. Miraflor Kian Joros Rebagos	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 14.1 Algorithm Complexity

1. PROCEDURE OUTPUT

What procedures did you perform?

	Best Case	Worst Case	Analysis
Algorithm 1	**O(n log2 n)**	**O(n log2 n)**	**Merge Sort demonstrates a consistent time complexity of O(n log2 n) across all cases.**
Algorithm 2	**O(n log2 n)**	**O(n^2)**	**Quick Sort relies on a partitioning mechanism (using the last element as the pivot). Its performance varies significantly based on how evenly the input array is divided**

Table 14-1. Best- and Worst-Case Analysis using Theoretical Tools

	Algorithm 1	Algorithm 2
Best Case	**Runs in O(n log2 n) time for any input order. Uses extra memory for temporary vectors during merging.**	**Runs in O(n log2 n) time when data splits evenly around the pivot. Very fast due to in-place array swaps.**
Worst Case	**Runs in O(n log2 n) time. Performance remains consistent even on unsorted or reversed data.**	**Runs in O(n^2) time if the array is already sorted or reverse-sorted, because the last element is picked as the pivot**

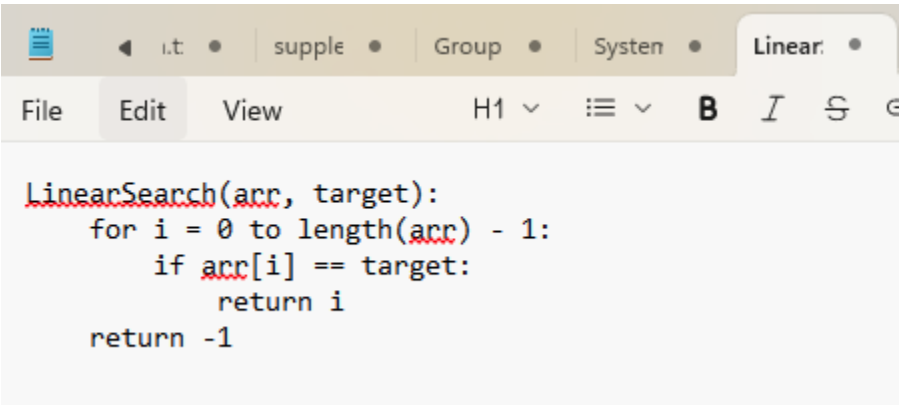
Table 14-2. Best- and Worst-Case Analysis using Experimental Tools

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

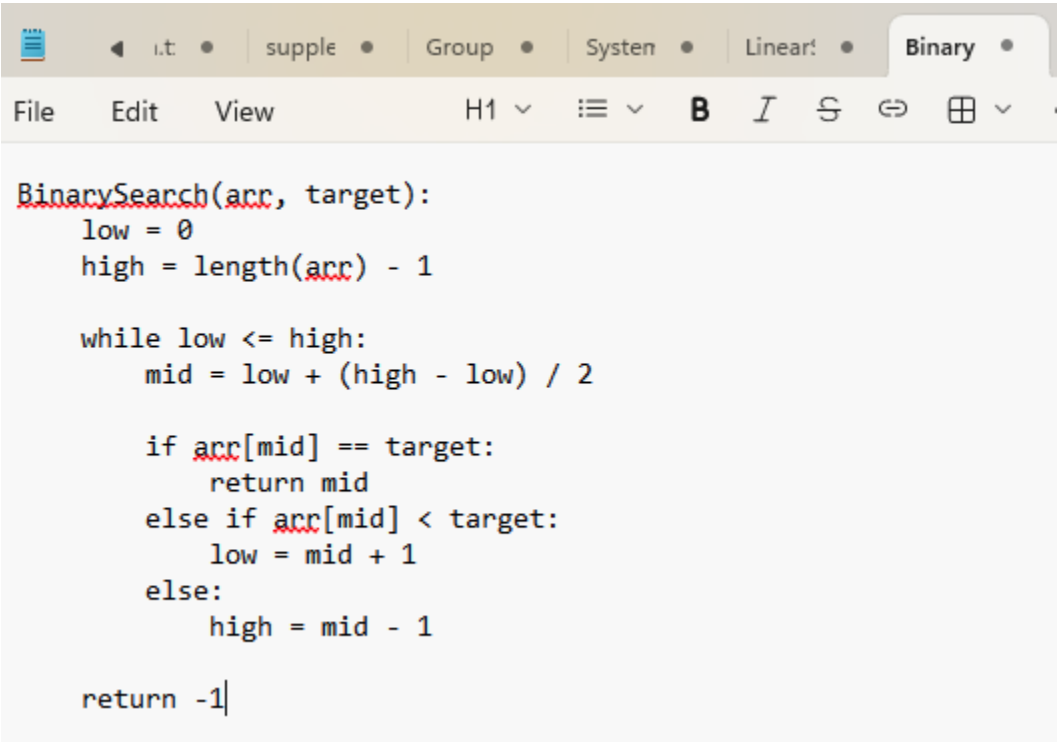
ILO A: Pseudocode Analysis

Linear Search:



Analysis: Linear Search checks each elements in the array sequentially from start to end until it finds the target value or reaches the end of the list.

Binary Search:



Analysis: Binary search uses a divide-and-conquer strategy. It compares the target with the middle

element and repeatedly cuts the search range in half.

Time and Space Complexities
Linear Search O(1)
Binary Search O(1)

Empirical Analysis

Input Size (N)	Elapsed Time for Binary Search (Worst-Case)	Elapsed Time for Linear Search (Worst-Case)
1,000	0.0001 ms	0.003 ms
10,000	0.0002 ms	0.025 ms
100,000	0.0004 ms	0.260 ms

Analysis:
Linear Search shows O(N) pattern is the worst-case scenario. Because it has to check every item one by one when an element is missing, multiplying the array size by 10 also multiplies the search time by about 10. For instance, searching through 1,000 items took about 0.003 ms, 10,000 items took 0.025 ms, and 100,000 items took 0.260 ms.

As for Binary Search it shows O(log N) pattern because it cuts the search area in half with every step. For 1,000 items, it only takes around 10 comparisons, and for 100,000 items, it only needs about 17 comparisons. That is why the execution time stays practically instant—rising from 0.0001 ms to just 0.0004 ms even after expanding the input size by 100 times.

Conclusion:
This HOA ACTIVITY demonstrates how algorithmic choice directly work, impacts performance as data scales. While Merge Sort guarantees consistent O(n log2 n) efficiency across all case, Quick Sort provides fast in-place operations but risks O(n^2) worst-case performance when handling pre-sorted data. Similarly, Binary Search's logarithmic approach (O(log n)) drastically outperforms Linear Search's O(n) strategy as input sizes grow, maintaining nearly instantaneous search times even for large datasets. Overall, selecting the appropriate algorithm depends heavily on memory constraints, input organization, and expected data volume.

This activity shows and taught me how choice of algorithm drastically impacts execution time as input size increases. For searching, Binary Search's O(log N) divide-and-conquer strategy outperforms Linear Search's O(N) sequential checking, staying nearly instantaneous (0.0004 ms at N=100,000) while Linear Search slows down linearly. For sorting, Merge Sort guarantees consistent O(n log2 n) performance regardless of initial array ordering, whereas Quick Sort is fast in practice but degrades to O(n^2) when given pre-sorted or reverse-sorted data due to its fixed pivot choice. Overall, understanding theoretical time complexity allows us to select the most efficient algorithm based on data size and initial array structure.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

COMPUTER ENGINEERING

QUEZON CITY

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.